

02443 Stochastic Simulation

Day 1: Generating Random Numbers

Nicolai Siim Larsen
Technical University of Denmark
DTU Compute
Section for Statistics and Data Analysis

Agenda

- 1 Introduction
- 2 Random numbers and RNGs

Agenda

1 Introduction

2 Random numbers and RNGs

Overview

In this lecture, we discuss the generation of pseudorandom numbers.

Key observations

- A large number of random number generators (RNGs) exist.
- The quality of RNGs varies considerably.
- Implementing your own RNG is generally not recommended outside research or educational settings.
- Built-in RNGs should always be evaluated before use.
- Modern scientific computing environments typically provide high-quality RNGs.
- Any RNG will eventually exhibit limitations under sufficiently demanding conditions.

Historical background

The need for random numbers has existed for centuries. Before computers, randomness was obtained using physical mechanisms or pre-generated tables.

Examples

- Tables of random numbers
- Lottery machines
- Roulette wheels
- Mechanical sampling devices

The development of computers created a need for algorithmic methods capable of generating large quantities of random numbers efficiently.

Methods for generating randomness

Mechanical devices

- Coin tosses
- Dice
- Roulette wheels
- Wheels of fortune
- Decks of cards
- Lottery machines

Physical devices

- Electronic noise in resistors or diodes
- Other physical noise sources

Pre-generated sources

- Tables of random numbers

Agenda

1 Introduction

2 Random numbers and RNGs

Random numbers

In stochastic simulation, we typically require a sequence

$$(U_1, U_2, U_3, \dots)$$

of independent and identically distributed random variables satisfying $U_i \sim U(0, 1)$.

Requirements

- Uniform distribution on the interval $(0, 1)$
- Statistical independence

A fundamental challenge is that computers operate with finite precision and therefore cannot generate truly random real numbers.

Definition of a random number generator

A random number generator (RNG) is a computer algorithm that produces a sequence of numbers that appears random.

Desired properties

- Uniform distribution on $(0, 1)$ or on a finite set of integers
- Statistical independence

Important remark

The phrase “appears random” means that the generated sequence should possess the same relevant statistical properties as a sequence of independent and identically distributed random variables.

Because computers use finite-precision arithmetic, exact uniformity on $(0, 1)$ can never be achieved.

The middle-square method

One of the earliest attempts at generating pseudorandom numbers was the middle-square method.

Algorithm

- 1 Start with a four-digit integer Z_i .
- 2 Square the integer.
- 3 Extract the middle four digits.
- 4 Use these digits as the next value Z_{i+1} .
- 5 Repeat.

i	Z_i	Z_i^2
0	7182	51,581,124
1	5811	33,767,721
2	7677	58,936,329
3	9363	87,665,769
4	6657	44,315,649
5	3156	09,960,336

Although the generated values may initially appear random, the method exhibits poor statistical properties and is unsuitable for practical simulation.

Fibonacci generators

The Fibonacci sequence is defined recursively by

$$x_i = x_{i-1} + x_{i-2}$$

with initial values $x_0 = 1$ and $x_1 = 1$.

This idea can be adapted to construct a random number generator.

Fibonacci generator

$$x_i = \text{mod}(x_{i-1} + x_{i-2}, M),$$

and

$$U_i = \frac{x_i}{M}.$$

The generated values satisfy

$$x_i \in \{0, 1, \dots, M-1\}.$$

The method is also known as an *additive congruential generator*.

Properties of Fibonacci generators

Recall that the modulus operation is defined by

$$x = \text{mod}(y, M) = y - \left\lfloor \frac{y}{M} \right\rfloor M.$$

Properties

- There are $M^2 - 1$ possible non-zero initial states.
- The generated sequence is periodic.
- The maximum possible period length is $M^2 - 1$.

Unfortunately, the maximal period is attained only in special cases ($M = 2$ or $M = 3$) and is generally much shorter in practice.

Congruential generators

A simple multiplicative generator can be defined by

$$U_i = \text{mod}(aU_{i-1}, 1), \quad U_i \in [0, 1).$$

The idea is that multiplication by a large constant a mixes the digits of the previous value.

Since computers work with integers, the generator is usually implemented as

$$x_i = \text{mod}(ax_{i-1}, M),$$

with

$$U_i = \frac{x_i}{M}.$$

Examples include choices such as

$$a = 23, \quad M = 10^8 + 1.$$

Cycles and periodicity

All pseudorandom number generators are deterministic. Consequently, once the initial state is fixed, the entire sequence is determined.

Important consequences

- Different initial states may lead to different cycles.
- Every generated sequence eventually repeats.
- The length of the cycle depends on the generator and its parameters.

If the state variable can assume N distinct values, then the cycle length can never exceed N .

Desirable properties of RNGs

A practical random number generator should possess several desirable properties.

Important criteria

- Long cycle length
- Good statistical properties
- Fast execution
- Reproducibility
- Portability across platforms

No generator is perfect, and the choice of RNG depends on the intended application.

Linear congruential generators

A linear congruential generator (LCG) is defined by

$$x_i = \text{mod}(ax_{i-1} + c, M),$$

where

- a is the multiplier,
- c is the increment,
- M is the modulus.

The corresponding pseudorandom numbers are obtained as

$$U_i = \frac{x_i}{M}.$$

The parameters should be chosen such that the sequence appears random and has a long period.

Example of an LCG

Consider the linear congruential generator

$$x_i = \text{mod}(5x_{i-1} + 1, 16),$$

with initial value

$$x_0 = 3.$$

The generated sequence becomes

$$(3, 0, 1, 6, 15, 12, 13, 2, 11, 8, 9, 14, 7, 4, 5, 10, 3, \dots)$$

The sequence returns to its starting value after 16 iterations and therefore has full period.

Full-period theorem for LCGs (Theorem 1)

A linear congruential generator achieves the maximal period length M if and only if the following conditions hold.

Conditions (Hull-Dobell)

- 1 M and c are relatively prime.
- 2 For every prime factor p of M , $a - 1$ is divisible by p i.e., $\text{mod}(a, p) = 1$.
- 3 If 4 divides M , $a - 1$ is divisible by 4 i.e., $\text{mod}(a, 4) = 1$.

Combining generators

One way to improve a random number generator is to combine several generators into a single sequence.

Examples

- XOR combinations of multiple generators
- Shuffling techniques
- Hybrid generators

Potential advantages

- Longer period lengths
- Improved statistical properties

However, the theoretical properties of combined generators may be difficult to analyse and verify.

The Mersenne Twister

The Mersenne Twister was introduced by Matsumoto and Nishimura in 1998 and has become one of the most influential pseudorandom number generators ever developed.

Characteristics

- Based on a large linear feedback shift register
- Uses 19,937 bits of internal state
- Extremely long period $2^{19937} - 1$
- Excellent statistical properties
- Efficient implementation

For many years, the Mersenne Twister was considered the standard choice for stochastic simulation.

Properties of the Mersenne Twister

The Mersenne Twister was designed to overcome several weaknesses of earlier generators.

Advantages

- Very long period length
- High-dimensional equidistribution
- Fast generation of random numbers
- Widely available in scientific software

Limitation

- Not suitable for cryptographic applications

For simulation purposes, however, it remains a highly reliable and widely used generator.

Random number generators in software

Modern scientific software packages typically provide high-quality random number generators by default.

Examples

- **R**: Mersenne Twister is the default generator, with many alternatives available.
- **Python**: Mersenne Twister is used by the standard `random` module.
- **MATLAB**: Mersenne Twister is the default generator in modern versions.

Most scientific computing environments therefore provide generators suitable for stochastic simulation without requiring users to implement their own methods.

What makes a good RNG?

Recall that a pseudorandom number generator produces a deterministic sequence $(U_1, U_2, U_3, \dots)$ that mimics the behaviour of independent random variables.

Important properties

- Correct probability distribution
- Statistical independence
- Long period length
- Computational efficiency
- Reproducibility

The challenge is to determine which statistical properties are most important for a given simulation study.

Summary

In this lecture, we have discussed the generation of pseudorandom numbers.

Main points

- Computers cannot generate truly random numbers.
- Random number generators produce deterministic sequences.
- Good generators mimic the behaviour of i.i.d. random numbers.
- Cycle length and statistical quality are essential considerations.
- Modern scientific software typically provides high-quality RNGs.
- The Mersenne Twister has been one of the most widely used RNGs for stochastic simulation.

The quality of a simulation study depends critically on the quality of the underlying random number generator.